RAPPORT TECHNIQUE — ARCHITECTURE VERTICALE D’AGRÉGATION

Nano-robots, essaim, méta-essaim

Les trois échelles d’EFE et comment l’information **remonte** : de la ligne de tenseur d’un nano-agent, au centroïde d’un essaim, au méta-consensus d’un essaim d’essaims — avec la maturité **réelle** de chaque étage.

Programme	EFE → Kovant · dossier de preuve (« dot »)
Périmètre	monorepo Rust efe · HEAD 665ead0 (+ working tree) · noyau efe-core + méta-essaim
Date	14 juillet 2026
Auteur	R. Pekar — avec Claude Code (Opus 4.8)
Méthode	lecture chirurgicale du code (citations fichier:ligne) + revue d’honnêteté
Règle cardinale	NE JAMAIS SUR-VENDRE — un nano-agent est une ligne de tenseur, pas un robot autonome

Le mot juste, d’emblée

Le code dit **nano-agent** ; « nano-robot » est le vocabulaire de présentation. À chaque échelle, l’« agent » et « l’information échangée » changent de nature : un nano-agent est une **ligne** d’un tenseur latent (sa voix est un vecteur) ; un essaim est cette population repliée en **un centroïde** ; un méta-essaim couple des essaims entiers, chacun vu comme un opérateur Φ. Ce rapport suit l’information qui **remonte** ces trois échelles, et dit à chaque étage ce qui est **servi**, **réel-mais-désactivé**, ou **stub**.

1 Résumé exécutif

EFE agrège l’information par **repli successif** sur trois échelles. À chaque étage, une population se réduit à un unique vecteur, qui devient l’unité de l’étage supérieur. C’est élégant — et il faut le décrire sans l’enjoliver.

Échelle 1 — le nano-robot (nano-agent). Concrètement, c’est une **ligne d’un tenseur latent** `[N,384]` en bf16, plus une étiquette de gabarit (u8), un identifiant et une saillance (f32). Il n’a **ni comportement, ni autonomie, ni mémoire locale, ni messagerie**. Ce qui distingue l’agent i de l’agent j : son gabarit (quel *prior* il copie) et son tirage aléatoire déterministe. C’est réel et servi (efe-server Mode-1) — mais c’est une ligne de tenseur, pas un robot.

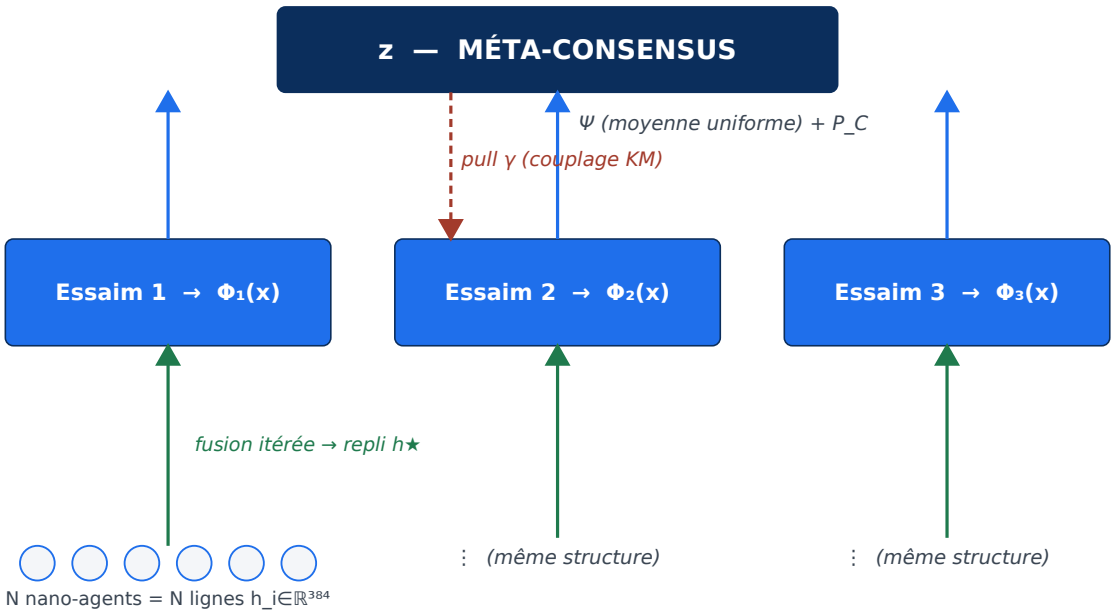
Échelle 2 — l’essaim (swarm). Un `SwarmController` orchestre la population comme une **machine à états linéaire** : genèse → fusion itérée (attention softmax N×N) → détection de consensus → alignement (projection sur un cône, avec porte de rupture qui **avorte** la requête) → dissolution en **un centroïde** (la moyenne de Fréchet). Ce centroïde h★ **est** la réponse de l’essaim. Réel et servi. Mais c’est un pipeline de stratégies fixe — **pas** une auto-organisation émergente — et la convergence est au mieux *best-effort* (un plancher de rondes peut retarder, jamais garantir, la convergence).

Échelle 3 — le méta-essaim (meta-swarm). Un essaim **entier** devient un opérateur Φ ; le contrôleur couple M participants par un point fixe de Krasnoselskii–Mann, en faisant circuler un méta-consensus comme *tirage*. Le transport gRPC/mTLS et l’itération couplée sont servis — mais le participant **par défaut** est un **mock** $\Phi(x)=\kappa\cdot x$, agrégé par une moyenne uniforme sans robustesse. Le vrai participant efe-core (un essaim servi comme Φ), la sonde de Lipschitz H1 et les bornes mesurées sont **réels mais désactivés par défaut** (`core-integration`). **C’est l’échelle où se situe tout notre travail M1.**

> **Trois retraits d’honnêteté à ne jamais oublier**
>
> (1) **Il n’y a pas de « nano-robot de sécurité qui vetote le contenu nuisible ».** Les comportements par gabarit (noyau de fusion, poids de consensus) sont **non câblés** ; le vote de refus poids-2 de `NaSafety` n’est **pas appliqué**, et `NaSafety` n’est même pas dans la population servie. (2) Les *priors* de gabarit servis sont **sémantiquement vides** (`scale·sin(i)`, « Not meaningful ») — le signal porteur est le *problem embedding* injecté (un vrai vecteur sentence-transformer) + les ancres du catalogue, pas les gabarits. (3) La population est **fixée à la genèse** : aucun agent n’est créé ni tué à l’exécution sur le chemin servi.

2 Le principe vertical — l’agrégation par repli

Le fil conducteur d’EFE est une **cascade de réductions** : N lignes de tenseur se replient en 1 centroïde d’essaim ; M centroïdes d’essaims se replient en 1 méta-consensus. Chaque échelle transforme une *population* en un unique *vecteur*, qui devient l’atome de l’échelle du dessus. Le schéma ci-dessous donne la vue d’ensemble ; les sections suivantes détaillent chaque étage, code à l’appui.



Le vertical d'EFE. En bas : les nano-agents (lignes du tenseur latent) échangent par attention softmax N×N et se replient (fusion itérée → centroïde h★) en la réponse d'un essaim. Au milieu : chaque essaim, exposé comme opérateur Φ, est un participant du méta-essaim. En haut : le contrôleur agrège les Φ (moyenne uniforme Ψ) et projette sur la sécurité (P_C) pour former le méta-consensus z, renvoyé aux participants comme tirage γ (couplage de Krasnoselskii–Mann).

Échelle	« Agent » =	Ce qui remonte	Opérateur de repli	Statut par défaut
1 · Nano-robot	ligne de tenseur h_i	un vecteur $\mathbb{R}^{384}$	attention softmax → agrégat	**SERVI**
2 · Essaim	population repliée	centroïde h★ (réponse)	moyenne de Fréchet (extract_centroid)	**SERVI**
3 · Méta-essaim	essaim = Φ	méta-consensus z	Ψ moyenne uniforme + projection P_C	**MOCK** κ·x

3 Échelle 1 — le nano-robot (nano-agent)

Statut : **PROD · servi** (dans efe-server Mode-1).

Ce qu'un nano-agent EST. À la genèse (`genesis/spawn_kernel.rs:107`), la population est un Structure-of-Arrays : un tenseur latent `[N,D]` en bf16, un vecteur de gabarits `[N] u8`, les *priors* `[K,D]`, et un masque `[N] u8`. Pour chaque agent i : h_i = prior[gabarit$_i$] + N(0,1)·σ (bruit tiré d'un `Pcg64` par-essaim), et masque[i]=1. Les seules métadonnées par-agent (`NanoAgentBatch`) sont : le gabarit, un `NanoAgentId`, et une saillance f32 (initialisée à 1.0, modifiée par la fusion). **Il n'y a aucune identité au-delà d'une ligne de tenseur + une étiquette + un scalaire** : pas de comportement, pas de but, pas de mémoire, pas de message.

Ce qui distingue un agent d'un autre. Exactement trois choses : son **index de gabarit** (quelle ligne de *prior* il copie), sa **ligne** dans le tenseur, et son **tirage aléatoire** (déterministe depuis le seed de l'essaim et la position). Le masque exclut une ligne inactive de l'attention ; à la genèse il est tout à 1. La « voix » d'un agent est sa ligne, lue par tous les autres à chaque ronde via l'attention.

La « mort » = la dissolution. Il n'y a pas d'événement de mort par-agent. La population « se dissout » quand `extract_centroid` replie toutes les lignes actives en **un seul vecteur de consensus** (la moyenne pondérée/de Fréchet). C'est une **réduction** au niveau tensoriel (N lignes → 1 vecteur), pas N agents qui « se mettent d'accord ». Les agents ne sont libérés que par RAII quand le batch et

l’ELS sont détruits.

> **Les garde-fous de l’échelle nano (contre la sur-vente)**
> • **Priors de gabarit vides** — le registre servi (`StubTemplateRegistry`) produit `prior = scale·sin(i)`, explicitement « Not meaningful — just distinguishable ». Le signal porteur est le *problem embedding* injecté (vrai vecteur all-MiniLM-L6, gardé dominant β_0=0.45) + les ancres du catalogue. • **Comportements par gabarit non câblés** — `NanoAgentTemplate` déclare `fusion_kernel/consensus_weight/centroid_init`, mais seul `default_share()` est consommé. Le vote de refus poids-2 de `NaSafety` n’est **pas appliqué**, et `NaSafety` n’est pas dans la population MVP (NaSem/NaLog/NaCrit seuls). **Ne pas présenter un nano-agent de sécurité opérant.** • **Population fixe** — `respawn_diversify` est un no-op ; la désactivation par masque (`collapse::PartialRespawn`) existe mais n’est **pas câblée**. Aucun spawn/kill dynamique servi. • **CPU seul** — le noyau de naissance CUDA est « Sprint 3+ ».

4 Échelle 2 — l’essaim (swarm)

Statut : **PROD · servi** (SwarmController, le vrai moteur Mode-1).

Un essaim est un `SwarmController` (`swarm/controller.rs:452`) qui ne voit le monde qu’à travers **cinq stratégies enfichables** (`swarm/strategies.rs`) : *Genesis* (naissance), *Fusion* (l’échange), *ConsensusDetector* (quand s’arrêter, avec état, réinitialisé par requête), *Dissolver* (extraire le centroïde + décoder) et *AlignmentProjector* (la porte de sécurité finale). L’ensemble des étages est **fixe** ; il n’y a pas de graphe reconfigurable.

4.1 · La machine à états (drive loop)

La boucle `drive()` (`controller.rs:452-634`) est un pipeline strictement linéaire : **(1) Genèse** → `(batch, els)` ; **(2) Fusion**, pour round 1..max_rounds : `run_round` puis `detector.evaluate` — un verdict *Converged* avant le plancher `min_rounds` est **rétrogradé en Continue** (le plancher D6) ; *Divergence* → erreur ; *ModeCollapse* → tentative de reprise (bruit + respawn) puis erreur ; aucune convergence après la boucle → `ConvergenceTimeout` ; **(3) Dissolution** : `extract_centroid` → h★, puis **alignement AVANT décodage** et porte de rupture ; **(4) Nettoyage** RAII. L’enum `SwarmState` (`Pending|Genesis|Fusion|Dissolution|Done|Failed`) est **observationnel** (posé sur le span de tracing) — une machine à états *documentée*, pas encore imposée par le typage (typestate = Phase-2).

4.2 · L’échange intra-essaim et le repli

L’échange est l’attention softmax N×N déjà décrite : chaque ligne se réécrit comme un mélange, pondéré par consensus, de toutes les autres, puis un mélange-β vers le *problem embedding* et une projection sur le cône. Le repli final (`extract_centroid`) est la moyenne (de Fréchet) des lignes actives : c’est la réponse de l’essaim, $h★ \in \mathbb{R}^{384}$. Sur le chemin servi, `PassThroughDissolver` renvoie le centroïde que la ronde de fusion a déposé dans un emplacement partagé (`LastCentroid`).

4.3 · La sécurité : le cône d’alignement et la porte de rupture

Le cône d’alignement (`AlignmentCone{anchors, max_projection_distance}`) porte les « priors constitutionnels ». Le projecteur strict (`dissolution/alignment.rs`) calcule les poids cosinus positifs du centroïde contre chaque ancre (négatifs annulés) ; s’ils sont tous nuls (orthogonal à toutes les ancres) → **rupture** (delta=∞) ; sinon il mélange `projected = (1-α)·raw + α·anchor_centroid` et, si `delta > rayon·(1+tolérance)` → **rupture**. Le contrôleur re-vérifie et, sur rupture, **avorte toute la requête** (jamais un centroïde en rupture n’atteint le décodeur). C’est un **échec dur**, pas un rognage-et-sert. L’alignement est **exigé et exécuté sur le chemin servi** (`AlignmentAdapter::strict_default`, α=0.5, tolérance=0.05).

> **Ce que l'essaim n'est PAS**
>
> • **Pas émergent / auto-organisé.** `drive()` est un pipeline fixe ; le seul élément adaptatif servi est la dynamique d'attention. Le mot « émancipation » ne désigne que la doctrine de fusion de la **bibliothèque**, que Mode-1 **atténue délibérément** (l'entrée reste dominante : β_0 0.20→0.45, α_0 0.50→0.25). • **Convergence non garantie.** `min_rounds` est un *plancher* qui peut retarder ou refuser la convergence, pas la garantir (`min_rounds > max_rounds` garantit même le timeout). • **Machine à états non typée** — imposée procéduralement en MVP.

5 Échelle 3 — le méta-essaim (meta-swarm)

Statut : transport **PROD · servi** · participant réel **réel · off par défaut** · défaut = **mock uniquement** (κ·x).

Un essaim entier devient un opérateur Φ. `SwarmController::phi_once(x)` (`controller.rs:412`) fixe `spec.problem_embedding = x` et `spec.max_rounds = 1`, puis exécute un `execute()` complet (genèse → **exactement UNE** ronde de fusion → dissolution → porte d'alignement → décodage pass-through) et renvoie le centroïde brut. C'est donc Φ(x) : x → embedding → une ronde → centroïde → retour dans l'espace de x. **Une ronde, PAS jusqu'à convergence** — et ce n'est pas un noyau léger : chaque appel paie la genèse (allocation d'arène, spawn), la dissolution et l'alignement complets. Le pas couplé enveloppe Φ dans le mélange de Krasnoselskii–Mann : `x_new = (1-α-γ)·current + α·Φ(current) + γ·pull`.

La boucle du méta-essaim. `CoupledIteration::run` (`iteration/coupled.rs`) fait, par pas : (1) *lift* parallèle de chaque état de participant vers l'espace méta ; (2) agrégation Ψ des votes → z ; (3) projection de sécurité `meta_safety.project(z)` ; (4) *pull* renvoyé à chaque participant ; (5) barrière parallèle `step_with_pull` ; (6) résidu. Arrêt sur résidu ≤ tolérance (Converged) ou max_iters. **Avant** d'itérer, H1 est pré-vérifié pour chaque participant et la boucle abandonne (`HypothesisCheckFailed`) si un certificat est rejeté. La sécurité est projetée **deux fois** par pas (sortie du participant + méta-consensus).

Mais le participant par défaut est un mock. `core-integration` est désactivée par défaut dans les deux crates ; sans elle, le seul participant est `MockParticipant` (Φ(x)=x ou κ·x), et l'agrégateur est `UniformMetaOp` (moyenne, borne de Lipschitz exactement 1, **aucune** robustesse aux aberrants). Le vrai pont efe-core (un essaim servi comme Φ) ne compile qu'avec la feature ; une ligne efe-core sur une build par défaut **échoue à la construction** (jamais un repli silencieux sur le mock).

> **H1 : un falsificateur, pas une preuve**
>
> La convergence repose sur H1 (Φ non-expansif, $L_\Phi \leq 1$). La sonde de Lipschitz (au boot, pour les moteurs efe-core) mesure une **borne inférieure** $\hat{L}$ par échantillonnage adverse : elle peut **rejeter** H1 ($\hat{L} \geq 1-\text{marge}$ ⇒ refus de servir, *fail-closed*), jamais la **certifier**. La topologie fixe rend Φ continu par construction (elle élimine la discontinuité du routage génésique), mais la contraction reste **property-testée, NON certifiée**. Mesuré : $\hat{L} \approx 0.37$ (fixe) / 0.69 (dynamique) — le certificat analytique $\beta_1 \approx 0.17$ est réfuté.

6 Le fil de l’agrégation — un vecteur, de bout en bout

Pour rendre la cascade concrète, suivons un vecteur depuis une ligne de nano-agent jusqu’au méta-consensus, en disant à chaque saut ce qui est réel et ce qui ne l’est pas :

Saut	Transformation	Réalité honnête
Naissance	h_i = prior[gabarit] + bruit·σ (une ligne $\mathbb{R}^{384}$)	Réel, servi. Mais le *prior* est du bruit distinguable, pas du savoir.
Échange intra-essaim	N×N attention softmax → mélange-β → projection cône (itéré)	Réel, servi. Le *problem embedding* injecté domine (β_0=0.45).
Repli de l’essaim	extract_centroid : N lignes → 1 centroïde h★	Réel, servi. Réduction par moyenne de Fréchet ; pas de rejet d’aberrants.
Porte de sécurité	projection sur le cône ; rupture ⇒ requête avortée	Réel, servi, *fail-closed*. Échec dur, pas rognage.
Essaim → Φ	phi_once : h★ redevient un pas de l’opérateur Φ(x)	Réel mais **off par défaut** (core-integration). Une ronde, pas convergé.
Agrégation méta	Ψ = moyenne uniforme des Φ_k + projection P_C → z	Transport servi ; par défaut les Φ_k sont des **mocks** κ·x. Aucune robustesse.
Couplage KM	z renvoyé comme pull γ ; x_new = mix(current, Φ, pull)	Réel, servi. H1 pré-vérifié ; certificats de fil = stub (vérifié au boot seulement).

La leçon honnête de ce tableau : **la mécanique d’agrégation est réelle et cohérente de bout en bout**, mais le *contenu* qui remonte est mince par défaut (priors vides, participants mock), et la seule échelle réellement servie avec un contenu porteur est l’essaim unique dans efe-server, où le vrai *problem embedding* domine.

7 Ce que nous avons construit — rendre un vrai essaim servable comme Φ (M1)

Notre chantier porte sur l’échelle 3 : remplacer le mock κ·x par un **vrai essaim efe-core** comme opérateur Φ du méta-essaim, servi sur gRPC, chaque pas verrouillé *fail-closed*. Sept commits (8fcb3dc → 665ead0), du 13 au 14 juillet 2026.

Commit	Jalon	Livré (échelle 3)
8fcb3dc	M1.a — couture	Le pas méta appelle un vrai `SwarmController` via `phi_once` ; `step_with_pull` = mélange couplé KM.
720d07c	M1.b — Φ déterministe	`init_sigma=0` + seed depuis l’id de spec → Φ bit-reproductible (prérequis de la sonde).
a8f1798	M1.c — sonde H1	Estimateur adverse de Lipschitz (bisection-discontinuités), certificat empirique, garde stricte fail-closed.
c53a7f6	M1.d — SERVI	Pont efe-core câblé dans la couche gRPC (bloc moteur, config_hash, boot-gate, require_convergence). Défaut = mock.
b9a8fce	Palier-2 — topologie fixe	Φ continu par construction. Mesure : $\hat{L}$ 0.37 (fixe) / 0.69 (dyn) — $\beta_1 \approx 0.17$ réfuté comme certificat.
665ead0	Dette de revue	10 findings soldés : bug de bypass d’identité-moteur au reload + retraits de sur-vente (ratio ~1.9 confondu).

Vérifié avant commit (665ead0)

Suite par défaut : **20 binaires, 0 échec** (lib 368). Suite feature `core-integration` : **27 binaires, 0 échec** (e2e efe-core sur gRPC 5/5). Re-revue adverse à 4 lentilles : **0 finding confirmé**.

8 Le grand livre d’honnêteté (les trois échelles)

Revendication tentante	Réalité vérifiée dans le code
« Des nano-robots autonomes collaborent. »	**Non.** Un nano-agent est une ligne de tenseur + une étiquette + une saillance. Pas de comportement, pas de message. La collaboration = une attention (un matmul).
« Un nano-agent de sécurité vetote le contenu nuisible. »	**Non câblé.** Le vote de refus poids-2 de `NaSafety` n’est pas appliqué ; `NaSafety` n’est pas dans la population servie.
« L’essaim s’auto-organise (émergence). »	**Non.** Pipeline de stratégies fixe. Mode-1 **atténue** la doctrine d’émancipation (entrée dominante).
« Les agents naissent et meurent dynamiquement. »	**Non.** Population fixée à la genèse. `respawn_diversify` est un no-op ; `PartialRespawn` n’est pas câblé.
« Convergence garantie / prouvée. »	**Best-effort.** Échec = `ConvergenceTimeout`. H1 = falsificateur (borne inf Ł), jamais prouvé ; $\beta_1 \approx 0.17$ réfuté (0.37/0.69).
« Le méta-essaim fait tourner de vrais essaims par défaut. »	**Mock par défaut** (κ·x). Le vrai essaim-comme-Φ est off (`core-integration`) — c’est le travail M1.
« Agrégation robuste / pondérée entre essaims. »	**Moyenne uniforme seule** (`UniformMetaOp`). Un participant adverse déplace le consensus.
« Les priors de gabarit encodent du savoir. »	**Non.** `scale·sin(i)`, « Not meaningful ». Le savoir vient du *problem embedding* injecté + ancres.

À **créditer** (sous-estimer serait aussi malhonnête) : la machine à états `drive()` avec ses stratégies découplées et sa **porte de rupture d’alignement qui avorte la requête** ; le plancher de convergence D6 + le consensus resserré ; l’ingénierie du déterminisme de Φ (qui rend la sonde possible) ; l’itération couplée du méta-essaim avec double projection de sécurité, pré-vérification H1 *fail-closed*, et `ConvergenceFailed` strict ; et le pont efe-core à manifeste *fail-closed* (une ligne efe-core sans la feature échoue, ne se rabat jamais en silence sur le mock). La **machinerie d’honnêteté** est réelle, même là où le contenu est mince.

9 Ce qui reste — feuille de route honnête (par échelle)

- **Échelle 1** — donner du sens aux gabarits : câbler `fusion_kernel/consensus_weight` (dont le vote de refus de sécurité) et remplacer les priors `sin(i)` par de vrais priors ; sinon, cesser de décrire des « nano-robots spécialisés » et parler d’une population homogène pilotée par l’embedding.
- **Échelle 2** — passer la machine à états en *typestate* (Phase-2) ; câbler une vraie reprise après effondrement (aujourd’hui no-op) ou retirer la revendication d’auto-guérison ; pondération de centroïde robuste aux aberrants (`DistanceInverse`, « Sprint 8+ »).
- **Échelle 3** — servir le vrai participant par défaut (⇒ certificats **sur le fil**, jalon M4, pour que le mode strict ne brique pas le service) ; une agrégation robuste servie (médiane/trimmed) ; réconcilier l’écart $\beta_1 \approx 0.17$ vs Ł≈ 0.37 (mesure étage-par-étage) et re-mesurer fixe-vs-dynamique à **population appariée** (le confond).
- **Transverse** — lander proprement le correctif « Φ pur » (non commité) et honorer la pureté sous concurrence pour le participant Fixe par défaut (aujourd’hui pur seulement sous exécution single-flight).

10 Annexe — servi vs défauts de bibliothèque, et index

Le profil servi (efe-server Mode-1) resserre la bibliothèque

Réglage	Défaut bibliothèque	Servi Mode-1	Effet
`min_rounds`	0 (aucun plancher)	4	plancher de convergence (D6)
consensus (conf./hyst.)	0.50 / 2	0.65 / 3	consensus plus exigeant
fusion β_0 / α_0	0.20 / 0.50 (émancipation)	0.45 / 0.25	entrée dominante
`init_sigma`	0.02 (requête) / 0.0 (latent)	idem	Φ déterministe au niveau latent
participant méta	—	Mock κ·x ; efe-core off	défaut = mock

Crates clés du vertical

- **Nano + essaim** : `efe-core` (`swarm/controller.rs`, `genesis/`, `fusion/`, `els/`, `dissolution/`) ; servi via `efe-server/mode1.rs`.
- **Méta-essaim** : `efe-meta-swarm` (opérateurs, itération couplée, sonde H1) + `efe-meta-swarm-server` (participant servi, gRPC/mTLS, manifeste, garde de reload).
- **Tests** : nano/essaim ≈ 427 inline (grep) ; méta-essaim — suites M1 **re-exécutées cette session** (défaut 368 lib + 20 bins, feature 27 bins, 0 échec).

> **Note de méthode & réserve**
> Rapport ancré sur une lecture chirurgicale du code (citations fichier:ligne) et une revue d’honnêteté. Les comptes de tests nano/essaim sont des estimations statiques par grep ; seules les suites M1 (échelle 3) ont été re-exécutées cette session. « Nano-robot » est le vocabulaire de présentation ; le code dit « nano-agent » — une ligne de tenseur. En cas de doute, ce document choisit la formulation la plus basse.